Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Вариант 4

По дисциплине «Основы машинного обучения»

Тема: **«Введение в нейронные сети: построение многослойного перцептрона»**

Выполнил:

Студенты 3 курса

Группы АС-67

Кирик А. Д.

Проверил:

Крощенко А. А.

Брест 2026

Цель работы: построить, обучить и оценить многослойный **перцептрон (MLP)** для решения задачи классификации

Задача: распознать цифру от 0 до 9 (10 классов).
Архитектура:

- входной слой;
- один скрытый слой с 32 нейронами (**ReLU**);
- выходной слой с 10 нейронами (**Softmax**).

Эксперимент: сравните результаты с архитектурой, где два скрытых слоя по 32 нейрона. Улучшилась ли точность?

Код программы:

```
import torch
from torch import nn
from torch import optim

import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import pandas as pd

device = torch.accelerator.current_accelerator().type if
torch.accelerator.is_available() else "cpu" # type: ignore
print("Платформа обучения: ", device)

class MLP(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.linear_relu_stack = nn.Sequential(
            nn.Flatten(),
            nn.ReLU(),
            nn.Linear(in_features=64, out_features=32),
            # Второй слой нейронной сети
            # nn.ReLU(),
            # nn.Linear(in_features=32, out_features=32),
            nn.Linear(in_features=32, out_features=10),
            # nn.Softmax(dim=10) # не применяется для многоклассовой
классификации
        )
    def forward(self, x):
        return self.linear_relu_stack(x)


# Обучение нейронной сети для многоклассовой классификации
def solution() -> None:
    TARGET : str = 'target'
    digits_dataset : pd.DataFrame = pd.read_csv('digits.csv', sep=',')
    y = digits_dataset[TARGET].values.astype(np.int32)
```

```
    X_train, X_test, y_train, y_test = train_test_split(
                digits_dataset.drop(columns=TARGET),
                torch.as_tensor(y, dtype=torch.long),
                test_size=0.25,
                random_state=42
            )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    X_train_tensor = torch.tensor(X_train_scaled, dtype=torch.float32)
    X_test_tensor = torch.tensor(X_test_scaled, dtype=torch.float32)

    mlp = MLP().to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(mlp.parameters(), lr=0.001)

    epochs : int = 50#125
    print("="*45)
    print("\nОбучаемая модель: \n", mlp)
    print("="*45)
    print("\nЦикл обучения модели: ")
    print("\tКоличество эпох: ", epochs)
    print("\tОптимизатор(Adam): ")
    for name, val in optimizer.defaults.items():
        print("\t\t", name, " : ", val)
    print("\tФункция потерь(многоклассовая): ", criterion)
    print("="*45)
    # Цикл обучения нейронной сети
    for _ in range(epochs):
        mlp.train()
        y_pred = mlp(X_train_tensor)
        loss = criterion(y_pred, y_train)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # Оценка модели
    print("\nОценка модели нейронной сети: ")
    mlp.eval()
    with torch.no_grad():
        y_pred = mlp(X_test_tensor)
        predicted_classes = torch.argmax(y_pred, dim=1)

        print(classification_report(y_test, predicted_classes))

if __name__ == "__main__":
    solution()
```

Спецификация вывода:

Платформа обучения: <тип ускорителя обучения>

Обучаемая модель: <слои модели>

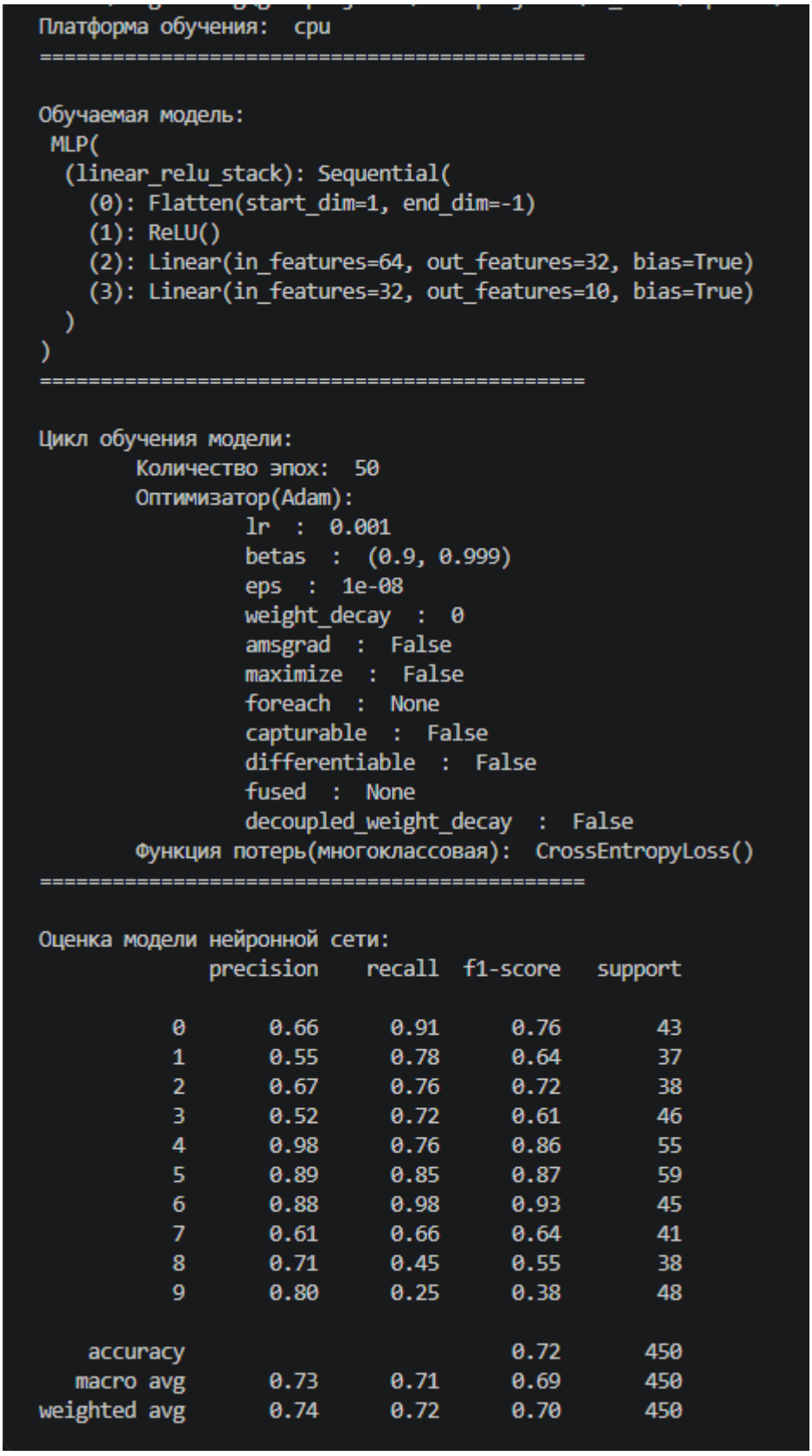
Цикл обучения модели:

Количество эпох: <количество эпох обучения>

Оптимизатор: <параметры оптимизатора>

Оценка модели нейронной сети: <оценка по accuracy, precision, recall, f1>

Графические материалы:



```
Платформа обучения:  cpu
========================================

Обучаемая модель:
 MLP(
  (linear_relu_stack): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): ReLU()
    (2): Linear(in_features=64, out_features=32, bias=True)
    (3): Linear(in_features=32, out_features=10, bias=True)
  )
)
========================================

Цикл обучения модели:
        Количество эпох:  50
        Оптимизатор(Adam):
                lr  :  0.001
                betas  :  (0.9, 0.999)
                eps  :  1e-08
                weight_decay  :  0
                amsgrad  :  False
                maximize  :  False
                foreach  :  None
                capturable  :  False
                differentiable  :  False
                fused  :  None
                decoupled_weight_decay  :  False
        Функция потерь(многоклассовая):  CrossEntropyLoss()
========================================

Оценка модели нейронной сети:
              precision    recall  f1-score   support

           0       0.66      0.91      0.76        43
           1       0.55      0.78      0.64        37
           2       0.67      0.76      0.72        38
           3       0.52      0.72      0.61        46
           4       0.98      0.76      0.86        55
           5       0.89      0.85      0.87        59
           6       0.88      0.98      0.93        45
           7       0.61      0.66      0.64        41
           8       0.71      0.45      0.55        38
           9       0.80      0.25      0.38        48

    accuracy                           0.72       450
   macro avg       0.73      0.71      0.69       450
weighted avg       0.74      0.72      0.70       450
```

Рис. 1. Результат программы с обучением нейронной сети с одним скрытым слоем на 32 нейрона (50 эпох)

```
Платформа обучения:  cpu
==========================================

Обучаемая модель:
 MLP(
  (linear_relu_stack): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): ReLU()
    (2): Linear(in_features=64, out_features=32, bias=True)
    (3): ReLU()
    (4): Linear(in_features=32, out_features=32, bias=True)
    (5): Linear(in_features=32, out_features=10, bias=True)
  )
)
==========================================

Цикл обучения модели:
        Количество эпох:  50
        Оптимизатор(Adam):
                lr  :  0.001
                betas  :  (0.9, 0.999)
                eps  :  1e-08
                weight_decay  :  0
                amsgrad  :  False
                maximize  :  False
                foreach  :  None
                capturable  :  False
                differentiable  :  False
                fused  :  None
                decoupled_weight_decay  :  False
        Функция потерь(многоклассовая):  CrossEntropyLoss()
==========================================

Оценка модели нейронной сети:
              precision    recall  f1-score   support

           0       0.66      0.67      0.67        43
           1       0.62      0.84      0.71        37
           2       0.70      0.84      0.76        38
           3       0.58      0.78      0.67        46
           4       0.80      0.95      0.87        55
           5       0.92      0.59      0.72        59
           6       0.89      0.87      0.88        45
           7       0.58      0.85      0.69        41
           8       0.54      0.34      0.42        38
           9       0.76      0.27      0.40        48

    accuracy                           0.70       450
   macro avg       0.71      0.70      0.68       450
weighted avg       0.72      0.70      0.68       450
```

Рис. 2. Результат программы с обучением нейронной сети с двумя скрытыми слоями на 32 нейрона (50 эпох)

Вывод: в ходе лабораторной работы была обучена нейронная сеть с одним и двумя скрытыми слоями. При добавлении дополнительного скрытого слоя на 32 нейрона немного ухудшило результаты модели в общем.